Development Documentation

Greenfield Local Hub Web Application

Development Documentation

Table of Contents

Day 1 - Project setup, models, scaffolding and layout 2

Day 1 - Development error I faced 7

Day 2 - Home pages, contact form, seed data and roles 7

Day 2 - Development error I faced 12

Day 3 - Shop page, search, filters, sorting and pagination 13

Day 3 - Development error I faced 16

Day 4 - Shopping cart, checkout and orders 17

Day 4 - Development error I faced 21

Day 5 - Address book and loyalty rewards 22

Day 5 - Development error I faced 26

Day 6 - Producer dashboard and authorisation 26

Day 6 - Development error I faced 30

Day 7 - Styling and accessibility 30

Day 7 - Development error I faced 33

Day 8 - Testing and final changes 34

Day 8 - Development error I faced 37

Development Diary

I wrote this development diary to display the methods I used to build my Greenfield Local Hub web application. I have sectioned the diary in days so it is clear for the reader to understand what I added, what problems I had and how I fixed them successfully.

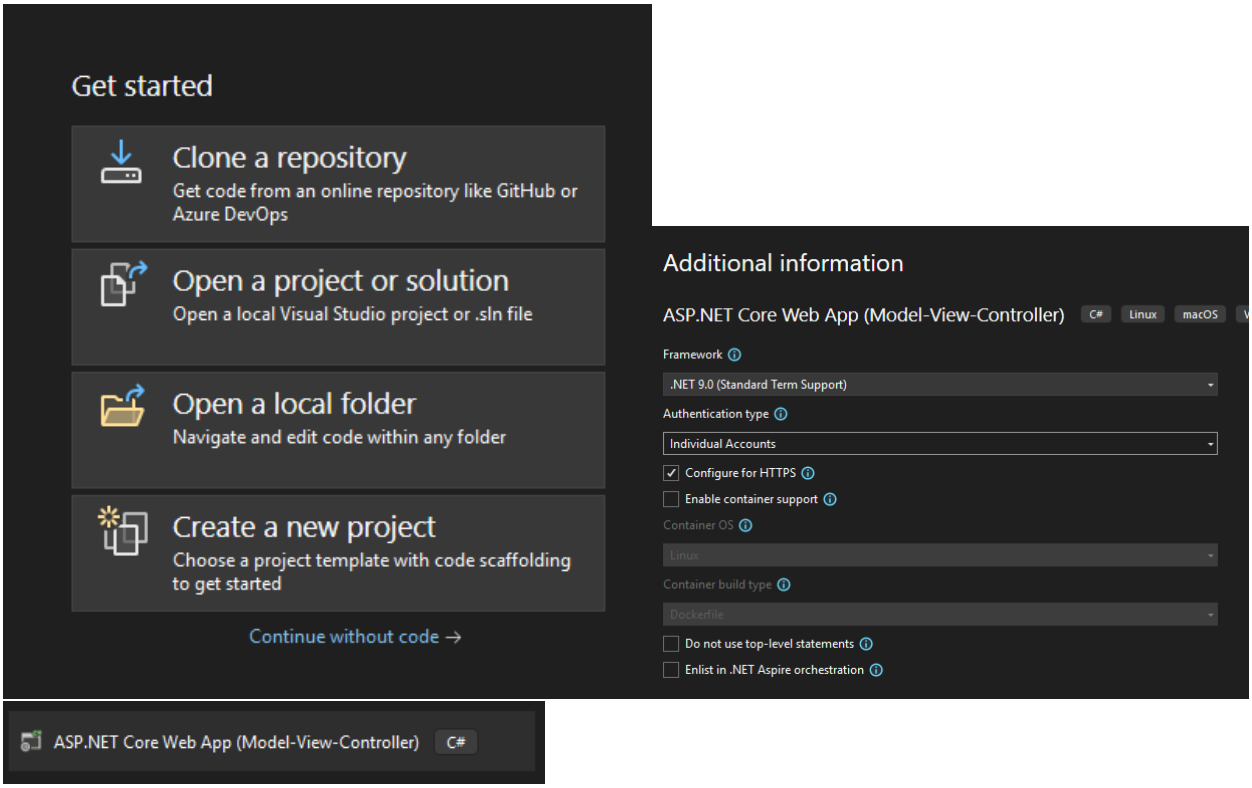
The digital solution is an ASP.NET Core MVC (Model-View-Controller) website for a local food cooperative. The purpose of this digital solution is to let customers browse local food, add products to cart, checkout by delivery or collection, use loyalty rewards and let producers manage their own products and orders.

Day 1 - Project setup, models, scaffolding and layout

1.1

I started by creating my project in Visual Studio using ASP.NET Core Web App (Model-View-Controller) in C#. I chose MVC because it keeps the project organised into Models, Views and Controllers. This helped me keep the business logic, database data and page design separated instead of mixing everything into one file.

I used .NET 9.0 because it is a newer framework and it gives the project the latest ASP.NET Core features. I also selected Individual Accounts because my website needs register, sign in, account pages and role based access for customers, producers, admins and developers.



I also checked the NuGet packages because the project needs Entity Framework Core, Identity, SQL Server and the scaffolding tools. These packages are important because they let the database, login system and generated MVC pages work properly.

Below you can see what .NET version is used and what frameworks that I have installed using the NuGet Package Manager in Visual Studio. This is important for my digital solution to run without errors and have all the functionality it is supposed to have.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>aspnet-GreenfieldLocalHubWebApp-93682f22-2d49-40dd-9050-de81b2c663c4</UserSecretsId>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Authentication.Google" Version="9.0.12" />
    <PackageReference Include="Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore" Version="9.0.0" />
    <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="9.0.12" />
    <PackageReference Include="Microsoft.AspNetCore.Identity.UI" Version="9.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="9.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.12">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.VisualStudio.Web.CodeGeneration.Design" Version="9.0.12" />
  </ItemGroup>

</Project>
```

1.2

After creating the project, I checked the default structure. I made sure the Controllers, Models, Views, Data, Areas and wwwroot folders were all created correctly. This helped me know where each part of the system should go before, I started adding my own code.

I then looked at Program.cs because this is where the services and middleware are set up. The database context, Identity, role support, MVC routes, Razor Pages and seed data are all connected from there. This was important because if startup is wrong then the rest of the website will not load correctly.

```
// Configure Identity with role support and account confirmation requirement
builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>() // Add support for roles
    .AddEntityFrameworkStores<ApplicationDbContext>();
// Register MVC controllers with views
builder.Services.AddControllersWithViews();
```

1.3

Then I created the main database models for the local food cooperative. I created models for products, producers, categories, shopping carts, shopping cart items, orders, order products, addresses, loyalty accounts and loyalty transactions. Each model represents one important part of the website.

I made the products model because the shop needs to show product name, description, price, stock, image, unit, category and producer. The navigation properties are used so the product can connect to its producer and category when the page loads.

```
public class products
{
    21 references
    public int productsId { get; set; }
    12 references
    public int producersId { get; set; }
    9 references
    public int categoriesId { get; set; }
    20 references
    public string productName { get; set; }
    16 references
    public string productDescription { get; set; }
    22 references
    public int stockQuantity { get; set; }
    23 references
    public float productPrice { get; set; }
    14 references
    public bool productAvailability { get; set; }
    16 references
    public string productImage { get; set; }
    0 references
    public string productUnit { get; set; }

    12 references
    public producers producers { get; set; }
    9 references
    public categories categories { get; set; }
    0 references
    public ICollection<orderProducts>? orderProducts { get; set; }
    0 references
    public ICollection<shoppingCartItems>? shoppingCartItems { get; set; }

}
```

I also created the orders model so checkout information can be stored properly. It stores whether the customer chose delivery or collection, the delivery fee, order status, order date, collection date and a delivery address.

```
public class orders
{
    18 references
    public int ordersId { get; set; }
    9 references
    public int? addressId { get; set; }
    10 references
    public string UserId { get; set; }
    9 references
    public float totalAmount { get; set; }
    11 references
    public bool delivery { get; set; }
    11 references
    public bool collection { get; set; }
    12 references
    public string? deliveryType { get; set; }
    0 references
    public float deliveryFee { get; set; }
    10 references
    public string orderStatus { get; set; }
    13 references
    public DateOnly? orderCollectionDate { get; set; }
    10 references
    public DateOnly orderDate { get; set; }
    9 references
    public string? DeliveryStreet { get; set; }
    9 references
    public string? DeliveryCity { get; set; }
    9 references
    public string? DeliveryPostalCode { get; set; }
    9 references
    public string? DeliveryCountry { get; set; }

    5 references
    public address? address { get; set; }
    4 references
    public ICollection<orderProducts>? orderProducts { get; set; }
}
```

1.4

After the models were created I linked them to ApplicationDbContext. This is important because Entity Framework will only create database tables for models that are registered as DbSet properties. Without this step the database would not know about my own models and that would create problems in the

future.

```
public class ApplicationDbContext : IdentityDbContext
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.address> address { get; set; } = default!;
    16 references
    public DbSet<GreenfieldLocalHubWebApp.Models.categories> categories { get; set; } = default!;
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.loyaltyAccount> loyaltyAccount { get; set; } = default!;
    7 references
    public DbSet<GreenfieldLocalHubWebApp.Models.loyaltyTransaction> loyaltyTransaction { get; set; } = default!;
    10 references
    public DbSet<GreenfieldLocalHubWebApp.Models.orderProducts> orderProducts { get; set; } = default!;
    10 references
    public DbSet<GreenfieldLocalHubWebApp.Models.orders> orders { get; set; } = default!;
    17 references
    public DbSet<GreenfieldLocalHubWebApp.Models.producers> producers { get; set; } = default!;
    16 references
    public DbSet<GreenfieldLocalHubWebApp.Models.products> products { get; set; } = default!;
    14 references
    public DbSet<GreenfieldLocalHubWebApp.Models.shoppingCart> shoppingCart { get; set; } = default!;
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.shoppingCartItems> shoppingCartItems { get; set; } = default!;
}
```

1.5

I then scaffolded my models to create the first controllers and Razor views. This saved time because it gave me create, edit, details, delete and index pages for the models. Immediately after scaffolding I added migration and updated the database to save all the data to the context (database). I will then in the future edit the scaffolded pages afterwards so they fit my own website instead of staying as basic generated pages, but I will have to do that in another day.

```
Add-Migration AddingAllModelsAsControllers
```
```
Update-Database
```

```
public class productsController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public productsController(ApplicationDbContext context)
    {
        _context = context;
    }

    3 references
    public async Task<IActionResult> Index()
    {
        var allProducts = await _context.products
            .Include(p => p.categories)
            .Include(p => p.producers)
            .ToListAsync();
        return View(allProducts);
    }
```

I also worked on the shared layout. I added the main navigation with Home, Shop, Producers, About Us, Contact, Cart and account links. The cart icon shows the number of items in the active cart, which makes the website easier to use because the user can see their basket from any page, but since I have not added any logic for that yet in the controllers it will have to stay as zero for now.

```
<a class="nav-link" role="listitem" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
<a class="nav-link" role="listitem" asp-area="" asp-controller="products" asp-action="Index">Shop</a>
<a class="nav-link" role="listitem" asp-area="" asp-controller="producers" asp-action="Index">Producers</a>
<a class="nav-link" role="listitem" asp-area="" asp-controller="Home" asp-action="About">About Us</a>
<a class="nav-link" role="listitem" asp-area="" asp-controller="Home" asp-action="Contact">Contact</a>
```

```
<!-- Cart icon -->
<a class="nav-icon-btn" asp-area="" asp-controller="shoppingCarts" asp-action="Index" aria-label="Shopping cart">
    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" aria-hidden="true">
        <circle cx="9" cy="21" r="1" />
        <circle cx="20" cy="21" r="1" />
        <path d="M1 1h4l2.68 13.39a2 2 0 0 0 2 1.61h9.72a2 2 0 0 0 2-1.61L23 6H6" />
    </svg>
    <span class="cart-badge" aria-label="items in cart">0</span>
</a>
```

I added a mobile navigation menu as well. This was important because users might use the website on phones and tablets, not just a computer.

Today I:

- Created the ASP.NET Core MVC project
- Used C# and .NET 9.0
- Selected Individual Accounts
- Checked the folder structure
- Created the main models
- Linked the models to ApplicationDbContext
- Added migrations and updated the database
- Scaffolded controllers and views
- Created the first shared layout and navigation

Tomorrow I want to:

- Create the home page and static pages
- Add the contact form
- Seed roles, users, producers, categories and products

Day 1 - Development error I faced

While I was setting up the project, I hit my first real Entity Framework problem. I had created the model classes, but when I tried to scaffold pages and query the database, the application did not know that those models were supposed to become database tables. It felt like the code existed, but the database layer could not see it yet.

Code section that gave the error:

```
public class ApplicationDbContext : IdentityDbContext
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

The error happened because I had model files such as products, producers and orders, but I had not registered them as DbSet properties. Entity Framework could not include them properly in the database context.

Current project code that fixes it:

```
public class ApplicationDbContext : IdentityDbContext
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.address> address { get; set; } = default!;
    16 references
    public DbSet<GreenfieldLocalHubWebApp.Models.categories> categories { get; set; } = default!;
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.loyaltyAccount> loyaltyAccount { get; set; } = default!;
    7 references
    public DbSet<GreenfieldLocalHubWebApp.Models.loyaltyTransaction> loyaltyTransaction { get; set; } = default!;
    10 references
    public DbSet<GreenfieldLocalHubWebApp.Models.orderProducts> orderProducts { get; set; } = default!;
    10 references
    public DbSet<GreenfieldLocalHubWebApp.Models.orders> orders { get; set; } = default!;
    17 references
    public DbSet<GreenfieldLocalHubWebApp.Models.producers> producers { get; set; } = default!;
    16 references
    public DbSet<GreenfieldLocalHubWebApp.Models.products> products { get; set; } = default!;
    14 references
    public DbSet<GreenfieldLocalHubWebApp.Models.shoppingCart> shoppingCart { get; set; } = default!;
    11 references
    public DbSet<GreenfieldLocalHubWebApp.Models.shoppingCartItems> shoppingCartItems { get; set; } = default!;
}
```

Once I added the DbSet properties, I could create migrations and the project started to feel like a real connected MVC application instead of just separate model classes.

Day 2 - Home pages, contact form, seed data and roles

2.1

I started this day by creating the Home, About Us, Contact, Privacy and Terms pages. I made these pages because a local food cooperative website needs a public front page, information about the business, a contact page and basic legal pages.

On the home page I displayed featured products and producer information. This helps the user understand quickly that the website is about local food and local producers. I loaded the products with producer data so the card could show where the item came from.

```
public async Task<IActionResult> Index()
{
    var producers = await _context.producers.OrderBy(p => p.producerName).ToListAsync();

    ViewBag.Producers = producers;

    var featuredProducts = await _context.products
        .Include(p => p.producers)
        .OrderByDescending(p => p.productsId)
        .Take(8)
        .ToListAsync();

    return View(featuredProducts);
}
```

2.2

I then created the contact form using a view model. I used a view model instead of a database model because the contact form is only used for sending an enquiry and it does not need to expose extra database fields. I added validation so the form will not submit if important fields are missing.

```
public class contactEnquiryViewModel
{
    [Required(ErrorMessage = "First name is required.")]
    1 reference
    public string FirstName { get; set; }

    [Required(ErrorMessage = "Last name is required.")]
    1 reference
    public string LastName { get; set; }

    [Required(ErrorMessage = "Email address is required.")]
    [EmailAddress(ErrorMessage = "Please enter a valid email address.")]
    3 references
    public string Email { get; set; }

    [Required(ErrorMessage = "Please select a subject.")]
    2 references
    public string Subject { get; set; }

    2 references
    public int? ProducerId { get; set; }

    [Required(ErrorMessage = "Please enter a message.")]
    [MinLength(10, ErrorMessage = "Message must be at least 10 characters.")]
    1 reference
    public string Message { get; set; }
}
```

2.3

I then created roles for Admin, Producer, User and Developer. I did this because different people should have different access. Customers should shop, producers should manage their own products and orders,

and admin/developer accounts should help with management and testing. So I created a SeedData.cs file in the Data folder where I will store all my seed data in the future. I start by making the seed roles and users method:

```
public static async Task seedRolesAndUsers(IServiceProvider serviceProvider, UserManager<IdentityUser> userManager, RoleManager<IdentityRole> roleManager)
{
    //All the Seeded roles
    string[] roleNames = { "Admin", "Producer", "User", "Developer" };
    foreach (string roleName in roleNames)
    {
        var roleExists = await roleManager.RoleExistsAsync(roleName);
        if (!roleExists)
        {
            var role = new IdentityRole(roleName);
            await roleManager.CreateAsync(role);

        }
    }
```

After the roles were created I seeded test users and assigned them to roles. This was useful because I could log in as different users and test the website as a customer, producer, admin and developer.

Admin user:

```
var adminUser = await userManager.FindByEmailAsync("admin@test.com");
if (adminUser == null)
{
    adminUser = new IdentityUser { UserName = "admin@test.com", Email = "admin@test.com", EmailConfirmed = true };
    await userManager.CreateAsync(adminUser, "Admin123!");
}

if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
{
    await userManager.AddToRoleAsync(adminUser, "Admin");
}
```

Producer users:

```
//Producer User 1
var producerUser = await userManager.FindByEmailAsync("producer@test.com");
if (producerUser == null)
{
    producerUser = new IdentityUser { UserName = "producer@test.com", Email = "producer@test.com", EmailConfirmed = true };
    await userManager.CreateAsync(producerUser, "Producer123!");
}

if (!await userManager.IsInRoleAsync(producerUser, "Producer"))
{
    await userManager.AddToRoleAsync(producerUser, "Producer");
}



//Producer User 2
var producerUser2 = await userManager.FindByEmailAsync("producer2@test.com");
if (producerUser2 == null)
{
    producerUser2 = new IdentityUser { UserName = "producer2@test.com", Email = "producer2@test.com", EmailConfirmed = true };
    await userManager.CreateAsync(producerUser2, "Producer123!");
}

if (!await userManager.IsInRoleAsync(producerUser2, "Producer"))
{
    await userManager.AddToRoleAsync(producerUser2, "Producer");
}
```

2.4

I seeded producers, categories and products so the website had real content when it started. I added local food products such as tomatoes, carrots, eggs, bread, meat, honey and preserves. This made the prototype look more complete and it also made testing much easier.

Seeded categories:

```
public static async Task seedCategories(IServiceProvider serviceProvider)
{
    var context = serviceProvider.GetRequiredService<ApplicationDbContext>();
    if (await context.categories.AnyAsync()) return;

    var list = new List<categories>
    {
        new categories { categoryName = "Fruit & Veg" },
        new categories { categoryName = "Dairy & Eggs" },
        new categories { categoryName = "Bakery" },
        new categories { categoryName = "Meat & Poultry" },
        new categories { categoryName = "Honey & Preserves" }
    };

    await context.categories.AddRangeAsync(list);
    await context.SaveChangesAsync();
}
```

Seeded products:

```
new products
{
    productName = "Heritage Tomatoes",
    productDescription = "Mixed heritage varieties",
    productPrice = 3.20f,
    stockQuantity = 100,
    productUnit = "kg",
    productAvailability = true,
    productImage = "/images/products/heritageTomatoes.jpg",
    producersId = greenAcreFarm.producersId,
    categoriesId = vegCat.categoriesId
},

new products
{
    productName = "Heritage Carrots",
    productDescription = "Sweet, colorful carrots",
    productPrice = 2.40f,
    stockQuantity = 80,
    productUnit = "bunch",
    productAvailability = true,
    productImage = "/images/products/heritageCarrots.jpg",
    producersId = greenAcreFarm.producersId,
    categoriesId = vegCat.categoriesId

},
```

I also connected the seed methods in Program.cs so the data is created when the application starts. In the future I plan to add seeded addresses, orders and loyalty accounts as well because they help the final prototype show more realistic account pages.

```
// Seed initial data into database on application startup
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    var userManager = services.GetRequiredService<UserManager<IdentityUser>>();
    var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
    var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    await SeedData.seedRolesAndUsers(services, userManager, roleManager);
    await SeedData.seedProducers(services);
    await SeedData.seedCategories(services);
    await SeedData.seedProducts(services);
}
```

Today I:

- Created the home page
- Created About Us, Contact, Privacy and Terms pages
- Displayed featured products on the home page
- Created a contact form with validation
- Added roles for Admin, Producer, User and Developer
- Seeded users and assigned them roles
- Seeded producers, categories and products
- Added more seed data for addresses, orders and loyalty accounts

Tomorrow I want to:

- Build the shop page
- Add product search, filters, sorting and pagination
- Make producer product lists only show their own products

Day 2 - Development error I faced

On this day I started working with Identity roles and seed users. The mistake I made was trying to assign users to roles before I had definitely created those roles. When I tested the seeded accounts, the role assignment did not behave properly because the role might not exist yet.

Code section that gave the error:

```
var adminUser = await userManager.FindByEmailAsync("admin@test.com");
await userManager.AddToRoleAsync(adminUser, "Admin");
```

This was too direct. If the Admin role had not been created first, the account could not be added to that role correctly.

Current project code that fixes it:

```
string[] roleNames = { "Admin", "Producer", "User", "Developer" };
foreach (string roleName in roleNames)
{
    var roleExists = await roleManager.RoleExistsAsync(roleName);
    if (!roleExists)
    {
        var role = new IdentityRole(roleName);
        await roleManager.CreateAsync(role);

    }
}
```

After this, I could seed the users and then assign them to roles with much more confidence. It also made testing easier because I knew the Admin, Producer, User and Developer accounts would start in the right state. But I had a problem with seeding the data after first try, and I had to research and found out the issue. Turns out when running the project once after seeding the first time, issues could come up since the data has already been seeded before so it rejects running the solution so data don't duplicate. So I found the method to delete/drop the database and updating it again, so old data gets removed and the new data will replace it when updating it.

```
Drop-Database
```

Day 3 - Shop page, search, filters, sorting and pagination

3.1

I worked on the shop page on this day. I displayed every product with its image, price, producer, category, stock level and add to cart button. This made the shop page the main place where customers can browse local food.

I also loaded categories into ViewBag so the page could show a category filter. This meant the user could filter by Fruit & Veg, Dairy & Eggs, Bakery, Meat & Poultry and Honey & Preserves.

```
public async Task<IActionResult> Index()
{

    ViewBag.Categories = await _context.categories.OrderBy(c => c.categoryName).ToListAsync();

    // Producers only see products that belong to them
    if (User.IsInRole("Producer"))
    {
        // Get the ID of the currently logged in user
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

        if(userId == null)
        {
            return Unauthorized();
        }

        // Find the producer profile that belongs to this user
        var producer = await _context.producers.FirstOrDefaultAsync(p => p.UserId == userId);

        if (producer == null)
        {
            return NotFound();
        }

        // Load all products owned by this producer
        var producerProducts = await _context.products.Where(p => p.producersId == producer.producersId).Include(p => p.producers).ToListAsync();
        return View(producerProducts);
    }
    else
    {
        // Other users see every product in the catalogue
        var allProducts = await _context.products.Include(p => p.categories).Include(p => p.producers).ToListAsync();
        return View(allProducts);
    }
}
```

3.2

I then added search logic. The search checks product name, product description, category name and producer name. This is better than only searching product names because users might search for a producer or type a category instead.

```
// Apply the search text across product, category and producer fields
if (!string.IsNullOrWhiteSpace(query))
{
    query = query.Trim().ToLower();

    productsQuery = productsQuery.Where(p =>
        p.productName.ToLower().Contains(query) ||
        p.productDescription.ToLower().Contains(query) ||
        p.categories.categoryName.ToLower().Contains(query) ||
        p.producers.producerName.ToLower().Contains(query));
}
```

At first the filters and sorting could remove each other. For example, if I filtered by category and then sorted by price, the category could be lost. I fixed this by keeping the current search and filter values in ViewBag and passing them back through the forms.

```
// Store current filter values to maintain state in the view
ViewBag.CurrentQuery = query;
ViewBag.CurrentCategoryId = categoryId;
ViewBag.CurrentStockFilter = stockFilter;
ViewBag.CurrentSort = sortBy;
ViewBag.CurrentPage = page;
```

3.3

I added category filters, stock filters and sorting. The stock filter lets the user view in stock, out of stock or low stock products. Sorting lets the user order products by price or name.

```
// Apply the selected category filter
if (categoryId.HasValue && categoryId.Value > 0)
{
    productsQuery = productsQuery
        .Where(p => p.categoriesId == categoryId.Value);
}
```

```
productsQuery = stockFilter switch
{
    "in_stock" => productsQuery.Where(p => p.stockQuantity > 0),
    "out_of_stock" => productsQuery.Where(p => p.stockQuantity == 0),
    "low_stock" => productsQuery.Where(p => p.stockQuantity <= 10 && p.stockQuantity > 0),
    _ => productsQuery
};
```

```
// Apply the selected sort order
productsQuery = sortBy switch
{
    "price_asc" => productsQuery.OrderBy(p => p.productPrice),
    "price_desc" => productsQuery.OrderByDescending(p => p.productPrice),
    "name_desc" => productsQuery.OrderByDescending(p => p.productName),
    "name_asc" => productsQuery.OrderBy(p => p.productName),
    _ => productsQuery.OrderBy(p => p.productName)
};
```

3.4

I added pagination so the shop does not show too many products on one page. I used 12 products per page because too many cards could make the page slower and harder to read. Pagination also makes the page look more organised.

```
var totalItems = await productsQuery.CountAsync();

var products = await productsQuery
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();

ViewBag.TotalItems = totalItems;
ViewBag.PageSize = pageSize;
ViewBag.TotalPages = (int)Math.Ceiling(totalItems / (double)pageSize);

return View("Index", products);
```

I also made the shop behave differently for producer accounts. If the user is a producer, the products page only shows products that belong to that producer. This is important because producers should not manage or see another producer's product management list.

```
// Producers only search products that belong to them
if (User.IsInRole("Producer"))
{
    // Find the producer profile that belongs to this user
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    var producer = await _context.producers
        .FirstOrDefaultAsync(p => p.UserId == userId);

    if (producer != null)
    {
        productsQuery = productsQuery
            .Where(p => p.producersId == producer.producersId);
    }
}
```

Today I:

- Created the shop page
- Displayed product cards with images, price, category and producer
- Added text search
- Added category filters
- Added stock filters
- Added sorting by price and name
- Added pagination with 12 products per page

- Kept filters and sorting working together
- Restricted producer product views to their own products

Tomorrow I want to:

- Create shopping cart functionality
- Redirect unauthorised users to sign in
- Create checkout with delivery and collection
- Reduce stock after an order is placed

Day 3 - Development error I faced

When I first added filtering and sorting to the shop, the page worked only one control at a time. I would filter by category, then choose a sort order, and suddenly the category filter disappeared. It was frustrating because the shop looked finished, but the user experience felt broken.

Code section that gave the error:

```
    <form method="get" asp-action="FilterProducts">
    <select name="sortBy" onchange="this.form.submit()">
        <option value="price_asc">Price low to high</option>
    </select>
</form>
```

This form submitted only the sort option, so it lost the current category and search values.

Current project code that fixes it:

```
ViewBag.CurrentQuery = query;
ViewBag.CurrentCategoryId = categoryId;
ViewBag.CurrentStockFilter = stockFilter;
ViewBag.CurrentSort = sortBy;
ViewBag.CurrentPage = page;
```

```
<!-- Preserve filters -->
<input type="hidden" name="categoryId" value="@ViewBag.CurrentCategoryId" />
<input type="hidden" name="stockFilter" value="@ViewBag.CurrentStockFilter" />
<input type="hidden" name="sortBy" value="@ViewBag.CurrentSort" />
```

By storing the current filter state and sending it back through hidden fields, the page began to behave like a proper shop. Search, filtering and sorting could work together instead of replacing each other.

Day 4 - Shopping cart, checkout and orders

4.1

I created the shopping cart functionality. Users can add a product from the shop page or product details page. If the user is not logged in, they are redirected to the sign in page instead of seeing a confusing error page.

```
// Get the ID of the currently logged in user
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

if (userId == null)
{
    return RedirectToPage("/Account/Login", new { area = "Identity" });
}
```

I made the cart create a new active cart if the user does not already have one. This means the user does not need to manually create a shopping cart. The system handles it when the first product is added.

```
var shoppingCart = await _context.shoppingCart
    .FirstOrDefaultAsync(c => c.UserId == userId && c.shoppingCartStatus == true);

if (shoppingCart == null)
{
    shoppingCart = new shoppingCart
    {
        UserId = userId,
        shoppingCartCreatedAt = DateTime.Now,
        shoppingCartStatus = true
    };
    _context.shoppingCart.Add(shoppingCart);
    await _context.SaveChangesAsync();
```

4.2

I added logic so if the product is already in the cart, the quantity increases instead of creating a duplicate line. I also stopped the quantity going above the available stock. This stops customers ordering more than the producer has available.

```
var shoppingCartItem = await _context.shoppingCartItems
    .FirstOrDefaultAsync(sc => sc.shoppingCartId == shoppingCart.shoppingCartId && sc.productsId == productsId);

if (shoppingCartItem != null)
{
    // Add the chosen quantity without exceeding available stock
    shoppingCartItem.quantity = Math.Min(
        shoppingCartItem.quantity + quantity,
        product.stockQuantity
    );
}
else
{
    // Add a new cart line for products that are not already in the cart
    shoppingCartItem = new shoppingCartItems
    {
        shoppingCartId = shoppingCart.shoppingCartId,
        productsId = productsId,
        quantity = Math.Min(quantity, product.stockQuantity)
    };
    _context.shoppingCartItems.Add(shoppingCartItem);
}
```

I added quantity buttons on the cart page. The user can increase or decrease the quantity. If the quantity becomes zero, the cart line is removed.

```
// Remove the cart item entirely if the quantity drops to zero
if (newQty <= 0)
{
    _context.shoppingCartItems.Remove(item);
}
else
```

4.3

I then created checkout. The user can choose delivery or collection. If delivery is selected, the user must choose a saved address and delivery type. If collection is selected, the user must choose a collection date.

I added controller validation because the checkout must stay safe even if someone changes the form in the browser. The system checks that either delivery or collection is chosen, and collection dates must be at least 2 days away.

```
if (!orders.collection && !orders.delivery)
    ModelState.AddModelError("delivery", "Please select either delivery or collection.");

// Collection requires a date that is at least 2 days from today
if (orders.collection)
{
    if (orders.orderCollectionDate == null)
        ModelState.AddModelError("orderCollectionDate", "Collection date is required.");
    else if (orders.orderCollectionDate.Value < DateOnly.FromDateTime(DateTime.Today.AddDays(2)))
        ModelState.AddModelError("orderCollectionDate", "Collection date must be at least 2 days from today.");
}
```

```
// Delivery requires a delivery type and a valid address that belongs to this user
if (orders.delivery)
{
    if (string.IsNullOrWhiteSpace(orders.deliveryType))
        ModelState.AddModelError("deliveryType", "Please select a delivery type.");

    if (!orders.addressId.HasValue || orders.addressId.Value <= 0)
        ModelState.AddModelError("addressId", "Please select a delivery address.");
```

4.4

I made the system calculate delivery fees and order totals. Delivery can be free if the customer has a free delivery offer or if the total is high enough. Otherwise the fee depends on the delivery type.

```
decimal deliveryFee = 0m;
if (orders.delivery)
{
    var loyaltyAccountForDelivery = await _context.loyaltyAccount
        .FirstOrDefaultAsync(l => l.UserId == userId);

    bool hasFreeDelivery = loyaltyAccountForDelivery?.PendingOffer == "Free Delivery";

    // Free delivery if the user has the offer or the cart total is £30 or more
    deliveryFee = hasFreeDelivery || cartTotalAfterDiscount >= 30m ? 0m :
        orders.deliveryType switch
        {
            "First Class" => 5.50m,
            "Next Day" => 7.99m,
            _ => 3.50m
        };
}

orders.deliveryFee = (float)deliveryFee;
orders.totalAmount = (float)(cartTotalAfterDiscount + deliveryFee);
```

4.5

When the order is placed, the system creates order products from the cart items and reduces stock. This is one of the most important parts because the product stock must change after a customer buys something.

```
// Save the order to the database
_context.orders.Add(orders);
await _context.SaveChangesAsync();


// Add each cart item as an order line and reduce the product's stock quantity
foreach (var item in cartItems)
{
    // Block the order if there isn't enough stock for any item
    if (item.products.stockQuantity < item.quantity)
    {
        ModelState.AddModelError("", $"Sorry, we only have {item.products.stockQuantity} units of {item.products.productName} in stock.");
        ViewBag.CartTotal = cartTotalAfterDiscount;
        ViewBag.SubtotalBeforeDiscount = subtotalBeforeDiscount;
        ViewBag.LoyaltyDiscount = loyaltyDiscount;
        ViewBag.shoppingCartId = shoppingCartId;
        return View(orders);
    }

    _context.orderProducts.Add(new orderProducts
    {
        ordersId = orders.ordersId,
        productsId = item.productsId,
        quantity = item.quantity,
        unitPrice = item.unitPrice
    });

    // Reduce stock so the purchased quantity is no longer available
    item.products.stockQuantity -= item.quantity;
}

// Mark the cart as inactive so it can no longer be used
cart.shoppingCartStatus = false;
await _context.SaveChangesAsync();
```

I also added order history and reorder. Customers can view their previous orders and add the same items back into the cart if there is enough stock. This makes returning customers faster because they do not need to search for every item again.

```
// Load all products that were included in the selected order
var orderItems = await _context.orderProducts
    .Where(op => op.ordersId == orderId)
    .ToListAsync();
```

```
// Add available order items to the cart and track anything that cannot be added
foreach (var item in orderItems)
{
    if (!products.TryGetValue(item.productsId, out var product) ||
        product.stockQuantity < item.quantity)
    {
        outOfStockItems.Add(product?.productName ?? "Unknown product");
        continue;
    }

    var existingCartItem = cartItems
        .FirstOrDefault(ci => ci.productsId == item.productsId);

    if (existingCartItem != null)
    {
        // Increase existing quantities without exceeding current stock
        var newQty = existingCartItem.quantity + item.quantity;
        existingCartItem.quantity = Math.Min(newQty, product.stockQuantity);
    }
    else
    {
        // Add a new cart line for products that are not already in the cart
        _context.shoppingCartItems.Add(new shoppingCartItems
        {
            shoppingCartId = cart.shoppingCartId,
            productsId = item.productsId,
            quantity = item.quantity,
            unitPrice = item.unitPrice
        });
    }
}
```

Today I:

- Created shopping cart functionality
- Redirected users to sign in if they are not logged in
- Created active carts automatically
- Merged duplicate cart items
- Added quantity increase and decrease
- Created checkout
- Added delivery and collection options
- Added checkout validation
- Calculated delivery fees and totals
- Created orders and order products
- Reduced stock after purchase
- Added order history and reorder

Tomorrow I want to:

- Create the address book
- Connect saved addresses to checkout
- Add loyalty accounts and reward offers

Day 4 - Development error I faced

The shopping cart gave me a very clear user experience problem. When I was not signed in and clicked Add to Cart, the application could send me to an unhelpful failed page. From a customer point of view that felt like the site had crashed, even though the real issue was simply that I needed to sign in.

```
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

if (userId == null)
{
    return NotFound();
}
```

That response was technically simple, but it did not guide the user to the next useful action.

Current project code that fixes it:

```
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

if (userId == null)
{
    return RedirectToPage("/Account/Login", new { area = "Identity" });
}
```

After changing this, the flow made much more sense. If I tried to shop before signing in, the website took me straight to the sign in page so I could continue as a real customer.

Day 5 - Address book and loyalty rewards

5.1

I created the address book so customers can save delivery addresses. I made the controller attach the address to the logged in user automatically instead of letting the user type a UserId. This is more secure and easier for the user.

```
// Get the ID of the currently logged in user
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

if (userId == null)
{
    return Unauthorized();
}

// Set fields that should never come from the form
address.UserId = userId;
ModelState.Remove("UserId");
```

I also added default address functionality. If the user only has one address, it becomes the default. If none of the user's addresses are default, the system promotes one. This stops checkout from having no sensible default address.

```
// Load this user's addresses with the default and newest addresses first
var addresses = _context.address
        .Where(a => a.UserId == userId)
        .OrderByDescending(a => a.IsDefault)
        .ThenByDescending(a => a.createdDate)
        .ToList();

// If the user only has one address, make it the default address
if (addresses.Count == 1 && !addresses[0].IsDefault)
{
    addresses[0].IsDefault = true;
    _context.address.Update(addresses[0]);
    await _context.SaveChangesAsync();
}
```

5.2

I linked saved addresses into checkout. Delivery orders need a selected address, but collection orders do not. When a delivery order is placed, the address fields are copied onto the order as a string. I did this so an old order still shows the delivery address even if the user edits or deletes the saved address later.

```
var userAddresses = await _context.address
    .Where(a => a.UserId == userId)
    .ToListAsync();

ViewBag.HasAddresses = userAddresses.Any();
```

```
if (orders.addressId.HasValue)
{
    var selectedAddress = await _context.address.FindAsync(orders.addressId.Value);
    if (selectedAddress != null)
    {
        orders.DeliveryStreet = selectedAddress.street;
        orders.DeliveryCity = selectedAddress.city;
        orders.DeliveryPostalCode = selectedAddress.postalCode;
        orders.DeliveryCountry = selectedAddress.country;
    }
}
```

5.3

I then created loyalty accounts and loyalty transactions. Customers earn points after placing orders. The loyalty account stores the points balance, tier, redeemed offers, active offers, consumed offers and pending offer.

```
// If the user has no loyalty account yet, create one at Bronze tier
if (loyaltyAccount == null)
{
    loyaltyAccount = new loyaltyAccount
    {
        UserId = userId,
        pointsBalance = 0,
        loyaltyTier = "Bronze",
        redeemedOffers = string.Empty,
        ActiveOffers = string.Empty
    };

    _context.loyaltyAccount.Add(loyaltyAccount);
    await _context.SaveChangesAsync();
```

I added points after checkout. The points are based on the order total. I also update the tier so the customer can move from Bronze to Silver, Gold or Platinum.

```
// Award 10 points for every £1 spent on the order
int pointsEarned = (int)(orders.totalAmount * 10);

_context.loyaltyTransaction.Add(new loyaltyTransaction
{
    loyaltyAccountId = loyaltyAccountFinal.loyaltyAccountId,
    ordersId = orders.ordersId,
    loyaltyPoints = pointsEarned,
    transactionType = "Earn",
    transactionDate = DateTime.Now
});
```

5.4

I added reward offers such as fruit and vegetable discount, free cheese, free delivery and a GBP 5 voucher. When a user redeems an offer, the points are deducted and the offer becomes active. It can then be applied in the cart.

```
var transaction = new loyaltyTransaction
{
    loyaltyAccountId = loyaltyAccount.loyaltyAccountId,
    loyaltyPoints = pointsCost,
    transactionType = "Redeem",
    transactionDate = DateTime.Now
};

loyaltyAccount.pointsBalance -= pointsCost;
loyaltyAccount.loyaltyTier = GetLoyaltyTier(loyaltyAccount.pointsBalance);
```

```
if (!activeList.Contains(offerTitle))
{
    activeList.Add(offerTitle);
    loyaltyAccount.ActiveOffers = string.Join(",", activeList);
}
```

I added checks so offers are one time use. After checkout the pending offer is removed from ActiveOffers and added to ConsumedOffers. This stops a reward being used multiple times.

```
// Remove from active and add to consumed
activeList.Remove(offerToConsume);

if (!consumedList.Contains(offerToConsume))
    consumedList.Add(offerToConsume);

loyaltyAccountFinal.ActiveOffers = string.Join(",", activeList);
loyaltyAccountFinal.ConsumedOffers = string.Join(",", consumedList);
loyaltyAccountFinal.PendingOffer = null;
```

I also added a transaction history page. This lets the user see their earned and redeemed points. I added pagination so the history page stays neat when there are lots of transactions.

```
// Apply pagination to the transaction list
int pageSize = 15;
int pageNumber = page ?? 1;
var transactions = await query
    .OrderByDescending(t => t.transactionDate)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

```
ViewBag.TotalCount = totalCount;
ViewBag.CurrentPage = pageNumber;
ViewBag.TotalPages = (int)Math.Ceiling(totalCount / (double)pageSize);
```

Today I:

- Created address book pages
- Linked addresses to the logged in user
- Added default address behaviour
- Connected saved addresses to checkout
- Copied delivery address snapshots onto orders
- Created loyalty accounts
- Added loyalty points and tiers
- Added redeemable offers
- Added active, pending and consumed offer tracking
- Added loyalty transaction history

Tomorrow I want to:

- Create the producer dashboard
- Let producers manage their own products
- Let producers view and update orders linked to their products

Day 5 - Development error I faced

While building the address book, I noticed a security and usability issue from the scaffolded style of form handling. Some forms can accidentally expose fields such as UserId, which is not something a customer should ever type or control. I wanted the website to decide ownership from the logged-in account.

Code section that gave the error:

```
[HttpPost]
0 references
public async Task<IActionResult> Create([Bind("addressId,street,city,postalCode,country")] address address)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (userId == null)
    {
        return Unauthorized();
    }

    address.UserId = userId;
    ModelState.Remove("UserId");

    if (ModelState.IsValid)
    {
        _context.Add(address);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    return View(address);
}
```

This made the address book feel more like a real account feature. The user enters the address, and the server quietly attaches it to the correct account.

Day 6 - Producer dashboard and authorisation

6.1

I created the producer dashboard so producers can manage their own products and see orders linked to their products. I restricted the controller to the Producer role because normal users should not access producer tools.

```
[Authorize(Roles = "Producer")]
1 reference
public class producerDashboardController : Controller
{
    private readonly ApplicationDbContext _context;
    0 references
    public producerDashboardController(ApplicationDbContext context)
    {
        _context = context;
    }
```

When the producer opens the dashboard, the system finds their producer record using the logged in UserId. This is important because the dashboard must be connected to the correct producer account.

```
// Get the ID of the currently logged in user
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

// Find the producer profile that belongs to this user
var producer = await _context.producers
    .FirstOrDefaultAsync(p => p.UserId == userId);

if (producer == null) return NotFound();
```

6.2

I added product management and summary information. The producer can see total products, low stock products and recent orders. This helps producers quickly see what needs attention without opening every page.

```
// Load all products owned by this producer
var products = await _context.products
    .Where(p => p.producersId == producer.producersId)
    .ToListAsync();
```

```
ViewBag.totalProducts = products.Count;
ViewBag.lowStockProducts = products.Count(p => p.stockQuantity < 5);
ViewBag.activeTab = activeTab ?? "products";
```

For product create, edit and delete I used role-based authorisation. Producers, admins and developers can manage products, but producers are forced to create products under their own producer account.

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Producer, Admin, Developer")]
0 references
public async Task<IActionResult> Create([Bind("productsId,producersId,categoriesId,productName,productDescription,stockQuantity,productPrice,productAvailability,productImage,
{
    // Remove navigation properties from ModelState so they don't cause false validation errors
    ModelState.Remove("producers");
    ModelState.Remove("categories");

    // For producers, always use their own producer ID instead of trusting the posted value
    if (User.IsInRole("Producer"))
    {
        // Get the producer profile for the logged in user
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var producer = await _context.producers
            .FirstOrDefaultAsync(p => p.UserId == userId);

        if (producer == null) return NotFound();

        products.producersId = producer.producersId;
    }

    // Remove producer ID from ModelState because it has been set server-side
    ModelState.Remove("producersId");

    if (ModelState.IsValid)
    {
        // Save the product to the database
        _context.Add(products);
        await _context.SaveChangesAsync();
        return RedirectToAction("Index", "producerDashboard", new { activeTab = "products" });
    }

    ViewBag.categoriesId = new SelectList(_context.categories, "categoriesId", "categoryName", products.categoriesId);
    ViewBag.producersId = new SelectList(_context.producers, "producersId", "producerName", products.producersId);
    return View(products);
}
```

6.3

I added order management to the producer dashboard. Producers can view orders that include their own products. They can also update the order status. The status changes differently depending on whether the order is for collection or delivery.

```
var order = await _context.orders
    .Include(o => o.orderProducts)
        .ThenInclude(op => op.products)
    .FirstOrDefaultAsync(o => o.ordersId == orderId &&
        o.orderProducts.Any(op => op.products.producersId == producer.producersId));

if (order == null) return NotFound();

order.orderStatus = order.collection
    ? order.orderStatus switch
    {
        "Pending" => "Processing",
        "Processing" => "Ready to Collect",
        "Ready to Collect" => "Collected",
        _ => order.orderStatus
    }
    : order.orderStatus switch
    {
        "Pending" => "Processing",
        "Processing" => "Dispatched",
        "Dispatched" => "Delivered",
        _ => order.orderStatus
    };

_context.Update(order);
await _context.SaveChangesAsync();
```

6.4

I also added authorisation to other pages that should not be open to normal users. For example, scaffolded management pages such as shopping cart details and product editing should be limited to admin, developer or producer roles where needed. This protects the website data.

```
// Shows the full details of a single shopping cart for admins and developers
[Authorize(Roles = "Admin, Developer")]
0 references
public async Task<IActionResult> Details(int? id)
{
    ViewBag.CartItemCount = await GetCartItemCount();

    if (id == null)
    {
        return NotFound();
    }

    var shoppingCart = await _context.shoppingCart
        .FirstOrDefaultAsync(m => m.shoppingCartId == id);
    if (shoppingCart == null)
    {
        return NotFound();
    }

    return View(shoppingCart);
}
```

Today I:

- Created the producer dashboard
- Restricted the dashboard to the Producer role
- Loaded the producer record using the logged in user
- Added product management for producers
- Added product management access for admins and developers
- Added low stock and product summary information
- Added producer order management
- Added order status updates
- Added more role based authorisation

Tomorrow I want to:

- Improve styling across the website
- Make pages feel like a local food cooperative
- Add accessibility tools
- Check the site works on smaller screens

Day 6 - Development error I faced

The producer dashboard created another important problem. At first, when a producer viewed products, it was too easy for the page logic to show products that belonged to other producers. That is not acceptable for a producer dashboard, because each producer should only manage their own catalogue.

Code section that gave the error:

```
var allProducts = await _context.products.Include(p => p.categories).Include(p => p.producers).ToListAsync();
return View(allProducts);
```

This loaded every product without checking which producer was signed in.

Current project code that fixes it:

```
// Producers only search products that belong to them
if (User.IsInRole("Producer"))
{
    // Find the producer profile that belongs to this user
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    var producer = await _context.producers
        .FirstOrDefaultAsync(p => p.UserId == userId);

    if (producer != null)
    {
        productsQuery = productsQuery
            .Where(p => p.producersId == producer.producersId);
    }
}
```

After this change, the dashboard started to behave like a proper role-based area. When I logged in as a producer, I saw my own products instead of the whole site catalogue.

Day 7 - Styling and accessibility

7.1

I worked on the styling for the website. I used CSS and Bootstrap to make the site look like a local food cooperative. I used green colours, cream backgrounds, clear cards, product images and buttons that stand out. This made the website feel more connected to fresh food and local producers.

```
:root {
    --green: #1B4D3F;
    --green-light: #2a6b58;
    --green-dark: #122e25;
    --green-faint: #f0f5f3;
    --brown: #7C5C3E;
    --brown-light: #9b7455;
    --white: #FFFFFF;
    --off-white: #F9F9F9;
    --border: #E2E2E2;
    --border-dark: #C8C8C8;
    --text: #1A1A1A;
    --text-muted: #6B6B6B;
    --text-light: #9A9A9A;
    --shadow-sm: 0 2px 8px rgba(0,0,0,.08);
    --shadow-md: 0 6px 20px rgba(0,0,0,.10);
    --font-serif: 'Playfair Display', Georgia, serif;
    --font-sans: 'DM Sans', system-ui, sans-serif;
    --transition: all .22s ease;
}
```

I created separate page CSS files for bigger pages. This helped keep the CSS organised because the product page, producers page, cart page, orders page, address page, loyalty page and dashboard all needed different layouts.

```
@section Styles {
    <link rel="stylesheet" href="~/css/pages/products/index.css" asp-append-version="true" />
}
```

7.2

I styled the product cards so the shop looks more professional. The cards show images, price, stock state, producer and the add to cart action. I added hover styling so the page feels more interactive.

```css
/* PRODUCT CARD */
.product-grid-gap {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
    gap: 1.5rem;
}

.product-card {
    background: var(--white);
    border: 1px solid var(--border);
    transition: var(--transition);
    cursor: pointer;
    overflow: hidden;
}
```

I also styled the shared navigation and footer which is in the _Layout.cshtml. This is important because the layout appears on almost every page. If the shared layout looks good, the whole site feels more consistent.

7.3

I added accessibility tools in the shared layout. The user can turn on high contrast, dark mode, read the page aloud, increase text size, decrease text size and reset the settings. I added these tools because more users should be able to use the website comfortably.

```html
<!-- Accessibility bar - provides quick controls for contrast, reading and text size -->
<div class="a11y-bar" role="toolbar" aria-label="Accessibility tools">
    <button class="a11y-btn" onclick="toggleHighContrast()">High Contrast</button>
    <button class="a11y-btn" onclick="toggleDarkMode()">Dark Mode</button>
    <button class="a11y-btn" onclick="toggleSpeech()" id="ttsBtn" aria-pressed="false">Read Page</button>
    <button class="a11y-btn" onclick="changeTextSize(1)">A+</button>
    <button class="a11y-btn" onclick="changeTextSize(-1)">A-</button>
    <button class="a11y-btn" onclick="resetAccessibility()">Reset</button>
</div>
```

```cpp
/* DARK MODE */
0 references
function toggleDarkMode() {
    const active = document.body.classList.toggle('dark-mode');
    localStorage.setItem('darkMode', active);
    showToast(active ? 'Dark mode enabled.' : 'Dark mode disabled.');
}

/* HIGH CONTRAST */
0 references
function toggleHighContrast() {
    const active = document.body.classList.toggle('high-contrast');
    localStorage.setItem('highContrast', active);
    showToast(active ? 'High contrast enabled.' : 'High contrast disabled.');
}

/* TEXT SIZE */
2 references
function applyFontSize() {
    document.documentElement.style.fontSize = fontSize + 'px';
    localStorage.setItem('fontSize', fontSize);
}

0 references
function changeTextSize(delta) {
    fontSize = Math.min(22, Math.max(14, fontSize + delta));
    applyFontSize();
    showToast('Text size: ' + fontSize + 'px');
}
```

```cpp
/* TEXT TO SPEECH */
0 references
function toggleSpeech() {
    const btn = document.getElementById('ttsBtn');

    if (isSpeaking) {
        speechSynthesis.cancel();
        isSpeaking = false;
        btn.setAttribute('aria-pressed', 'false');
        showToast('Speech stopped.');
        return;
    }

    const text = getReadableText();

    if (!text) {
        showToast('No readable content found.');
        return;
    }

    utterance = new SpeechSynthesisUtterance(text);
    utterance.lang = 'en-GB';
    utterance.rate = 1;
    utterance.pitch = 1;

    utterance.onend = function () {
        isSpeaking = false;
        btn.setAttribute('aria-pressed', 'false');
    };

    speechSynthesis.speak(utterance);

    isSpeaking = true;
    btn.setAttribute('aria-pressed', 'true');
    showToast('Reading page...');
}
```

I also added a skip link and ARIA labels. This helps keyboard and screen reader users move around the site. I used aria-labels on navigation and cart actions so the controls are clearer.

7.4

I checked the site on smaller screens by using the inspect toggle-device-toolbar on my browser while running the application. This matters because users might use the cooperative website while shopping or collecting food, so the site should still work well on a phone.

```html
        <button class="nav-hamburger" aria-label="Toggle navigation" aria-expanded="false" onclick="toggleMobileNav(this)">
            <svg viewBox="0 0 24 24" aria-hidden="true">
                <line x1="3" y1="6" x2="21" y2="6" />
                <line x1="3" y1="12" x2="21" y2="12" />
                <line x1="3" y1="18" x2="21" y2="18" />
            </svg>
        </button>
    </div>

</nav>
```

```
/* MOBILE NAV */
0 references
function toggleMobileNav(btn) {
    const nav = document.getElementById('mobile-nav');
    const open = nav.style.display === 'block';

    nav.style.display = open ? 'none' : 'block';
    btn.setAttribute('aria-expanded', !open);

    if (!open) {
        nav.querySelector('a')?.focus();
    }
}
```

Today I:

- Styled the website with CSS and Bootstrap
- Styled the navigation and footer
- Styled shop, cart, orders, address book, loyalty and producer dashboard pages
- Added page-specific CSS files
- Added high contrast mode
- Added dark mode
- Added read page text-to-speech
- Added text size controls
- Added skip link and ARIA labels
- Checked smaller screen layout

Tomorrow I want to:

- Finish testing
- Add missing test cases to the test log
- Review final pages and documentation

Day 7 - Development error I faced

When I worked on the layout and accessibility tools, the mobile navigation was one of the small problems that made the site feel unfinished. The menu could open, but it did not properly toggle closed or update the accessibility state for screen readers.

Code section that gave the error:

```
function toggleMobileNav() {
    const nav = document.getElementById("mobile-nav");
    nav.style.display = "block";
}
```

This only opened the menu. It did not know whether the menu was already open, and it did not update aria-expanded.

Current project code that fixes it:

```
/* MOBILE NAV */
0 references
function toggleMobileNav(btn) {
    const nav = document.getElementById('mobile-nav');
    const open = nav.style.display === 'block';

    nav.style.display = open ? 'none' : 'block';
    btn.setAttribute('aria-expanded', !open);

    if (!open) {
        nav.querySelector('a')?.focus();
    }
}
```

This was a small piece of JavaScript, but it made the navigation feel more complete. It worked visually and also gave assistive technology a clearer state.

Day 8 - Testing and final changes

8.1

On the final day I tested the website using my test log. I tested navigation links, shop filters, search, product details, producer pages, cart, checkout, orders, address book, loyalty rewards, account pages and authorisation.

I also tested validation because this is important for a shopping website. For example, checkout should not allow no fulfilment choice, no delivery address, invalid collection dates, or ordering more stock than is available.

```
// If the offer conditions aren't met, show an error and redisplay the form
if (!offerConditionMet)
{
    ModelState.AddModelError(string.Empty, offerWarning);

    var (sub, disc, total) = await CalculateOrderTotals(userId, shoppingCartId);
    ViewBag.CartTotal = total;
    ViewBag.SubtotalBeforeDiscount = sub;
    ViewBag.LoyaltyDiscount = disc;
    ViewBag.HasFreeDelivery = false;
    ViewBag.shoppingCartId = shoppingCartId;

    var userAddresses = await _context.address
        .Where(a => a.UserId == userId)
        .ToListAsync();
    ViewBag.HasAddresses = userAddresses.Any();
    ViewData["AddressId"] = new SelectList(
        userAddresses, "addressId", "street", orders.addressId);

    return View(orders);
}
```

```
// Block the order if there isn't enough stock for any item
if (item.products.stockQuantity < item.quantity)
{
    ModelState.AddModelError("", $"Sorry, we only have {item.products.stockQuantity} units of {item.products.productName} in stock.");
    ViewBag.CartTotal = cartTotalAfterDiscount;
    ViewBag.SubtotalBeforeDiscount = subtotalBeforeDiscount;
    ViewBag.LoyaltyDiscount = loyaltyDiscount;
    ViewBag.shoppingCartId = shoppingCartId;
    return View(orders);
}
```

8.2

I tested authorisation by logging in as different users. I checked that normal users could shop, producers could access the producer dashboard, and management actions were restricted to the correct roles. This was important because the website has customer data, producer tools and admin-style pages.

I also checked anti-forgery validation on form posts. This helps protect forms such as create, edit, delete, checkout, reorder and apply offer.

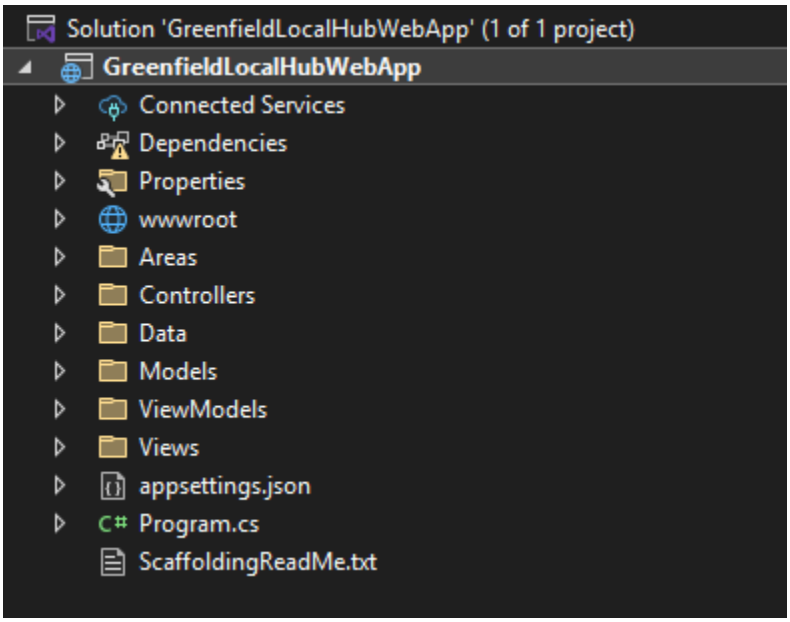
```
// Applies the selected loyalty offer to the user's active shopping cart
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> ApplyOffer(string selectedOffer)
{
    // Get the ID of the currently logged in user
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (userId == null) return Unauthorized();

    var loyaltyAccount = await _context.loyaltyAccount
        .FirstOrDefaultAsync(l => l.UserId == userId);

    if (loyaltyAccount == null) return NotFound();

    // Make sure the chosen offer is genuinely active before accepting it
    var activeOffers = string.IsNullOrEmpty(loyaltyAccount.ActiveOffers)
        ? new List<string>()
        : loyaltyAccount.ActiveOffers.Split(',').Select(s => s.Trim()).ToList();

    // Store the selected offer as pending, or clear it if the user selected none
    loyaltyAccount.PendingOffer = (selectedOffer == "none" || !activeOffers.Contains(selectedOffer))
        ? null
        : selectedOffer;

    _context.Update(loyaltyAccount);
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}
```

8.3

I added more tests for pages I had not tested before, such as Privacy, Terms, Contact form, Register, Sign in, account profile, address book, checkout validation, loyalty transaction history, producer dashboard and authorisation. This made the test log more complete.

I also checked that the account side panel active styling worked correctly. The profile page should not stay highlighted when the user is on orders, loyalty or address book. This is a small thing but it makes the account area feel more finished.

I checked the final project structure as well. The project has Controllers, Models, Views, ViewModels, Data, Areas, wwwroot, CSS, JavaScript, images and migrations. This showed that the project was organised and not just one page.



8.4

After testing, I made sure to comment my code and explain the main features and the code structure.

Today I:

- Tested the main user journeys
- Tested navigation links
- Tested shop search, filters, sorting and pagination
- Tested cart and checkout validation
- Tested address book and saved addresses
- Tested loyalty rewards and transaction history
- Tested producer dashboard
- Tested role based authorisation
- Added missing test cases to the test log
- Finished the final documentation

Day 8 - Development error I faced

During final testing I found that checkout needed stronger back-end validation. The page could look correct in the browser, but I still needed the controller to protect the order process if a user submitted missing or invalid fulfilment details.

Code section that gave the error:

```
orders.delivery = fulfilmentChoice == "Delivery";
orders.collection = fulfilmentChoice == "Collection";
```

This was not enough because it trusted that the form had been completed correctly.

Current project code that fixes it:

```
// Delivery orders don't need a collection date
if (orders.delivery)
{
    ModelState.Remove("orderCollectionDate");
}

if (!orders.collection && !orders.delivery)
    ModelState.AddModelError("delivery", "Please select either delivery or collection.");

// Collection requires a date that is at least 2 days from today
if (orders.collection)
{
    if (orders.orderCollectionDate == null)
        ModelState.AddModelError("orderCollectionDate", "Collection date is required.");
    else if (orders.orderCollectionDate.Value < DateOnly.FromDateTime(DateTime.Today.AddDays(2)))
        ModelState.AddModelError("orderCollectionDate", "Collection date must be at least 2 days from today.");
}

// Delivery requires a delivery type and a valid address that belongs to this user
if (orders.delivery)
{
    if (string.IsNullOrWhiteSpace(orders.deliveryType))
        ModelState.AddModelError("deliveryType", "Please select a delivery type.");

    if (!orders.addressId.HasValue || orders.addressId.Value <= 0)
        ModelState.AddModelError("addressId", "Please select a delivery address.");
```

```
return View(orders);
```

This was the point where testing really helped me. I was no longer just checking that the happy path worked; I was checking that the website protected the user from incomplete orders as well.